

6 — Observer: track on read, trigger on write

REACTIVITY TRACK TRIGGER

Q. An effect reads `claps()` exactly once. Later, `setClaps(2)` re-runs it. How did the write know the effect cared?

ANSWER. Because the read remembered. While an effect runs, a single global slot called `activeEffect` holds it. When the effect reads a signal, that signal records the effect in its `deps` set — that is *track on read*. When the signal is later written, it walks that set and re-runs every recorded effect — that is *trigger on write*.

WHY IT WORKS. `effect(fn)` parks `fn` in the slot, then runs it once so its reads can subscribe. Each signal it touches sees the slot occupied and adds `fn` to its dependents. The slot is cleared, but the subscriptions remain. A later write re-runs exactly those dependents and nobody else. Two moves — track and trigger — and you have reactivity.

examples/observer.js

```
let activeEffect = null; // who is
reading right now?
function signal(initial) {
  let value = initial;
  const deps = new Set();
  function read() {
    if (activeEffect) deps.add(activeEffect); //
    track on read
    return value;
  }
  function write(next) {
    value = next;
    deps.forEach(run => run()); //
    trigger on write
  }
  return [read, write];
}
function effect(fn) {
  activeEffect = fn; // park fn as
  the active reader
  fn(); // run once so
  its reads subscribe
```

```
    activeEffect = null;
  }
  const [claps, setClaps] = signal(0);
  effect(() => console.log("claps:", claps())); //
  prints "claps: 0"
  setClaps(1); //
  re-runs effect → "claps: 1"
  setClaps(2); //
  re-runs effect → "claps: 2"
```

THE ENGINE, TWO MOVES

A · TRACK

effect reads claps() → **signal records fn**

deps ← **fn** · parked in **activeEffect** while reading

B · TRIGGER

setClaps(2) writes → **deps re-run fn()**

"claps: 2" · every recorded effect, no one else

- ① **Summary.** A value that notices its readers and re-runs them when it changes is the whole engine of reactivity — track on read, trigger on write. The next step is to package that engine so plain dot-notation does both moves for you.